International Journal of Control, Automation, and Systems 22(5) (2024) 1613-1623
http://dx.doi.org/10.1007/s12555-022-1218-x
ISSN:1598-6446 eISSN:2005-4092
http://www.springer.com/12555

Quadrupedal Locomotion in an Energy-efficient Way Based on Reinforcement Learning

Tiantian Hao*, De Xu, and Shaohua Yan

Abstract: Achieving energy-efficient motion is important for the application of quadruped robots in a wide range. In this paper, we propose a hierarchical control framework that combines reinforcement learning and virtual model control to achieve energy-efficient motion with a planned gait. A reinforcement learning network is designed to learn the policy that maps the state of the robot to the action. The action is the increment of stance ratio, one of the gait parameters. The learned policy network is used as a high-level gait parameter modulator to adjust the gait parameters according to the body's velocity. The virtual model control method is used to compute the required force of robot's body. Then this force is decomposed to the feet of the stance legs with quadratic programming optimization. In the lowest level, the proportional-derivative controllers are used to control the joints' motion. Simulation and experiments are well conducted on the robot A1. The experimental results verify the effectiveness of the proposed method.

Keywords: Energy-efficient motion, quadruped robot, reinforcement learning, virtual model control.

1. INTRODUCTION

In practical application scenarios, speed command is often provided to the quadruped robot as motion control signals [1-3]. The quadruped robot usually moves with a fixed gait pattern at different speeds. Although the fixed gait pattern can be suitable for a range of speeds, the robot lacks the flexibility and adaptability to deal with variable speeds and may consume more energy during the motion [4]. In addition, the size and capacity of the on-board battery are often limited in order to ensure the flexibility of the movement. Thus, energy-efficient techniques must be considered to extend the robot's working hours. On the one hand, the lithium cells can be replaced with prospective perovskite solar cells which are high-power conversion efficiency, low cost and portable to supply enough energy for the robot [5]. On the other hand, it is necessary to consider how to make the quadruped robots move in an energy-efficient way, which is of great relevance to their future application. This article focuses on the latter. Some researches are inspired by the motion of the quadruped mammals which can adjust the gait pattern according to the speed [6-9]. In [8], the transition between the different gaits depending upon the speed of movement was predefined to enhance the efficiency of locomotion. In [9], a control policy using reinforcement learning (RL) learned multiple gaits and transitions while following the speed commands smoothly in natural environment. It improved the adaptability of the transition. But they did not consider the energy consumption during the motion.

It is difficult for the traditional planned approaches to generate the gait patterns with the smooth transition and the energy-efficient movement. Learning-based methods have the potential to improve the agility of quadruped robots [10-12]. It can give consideration to both the gait pattern learning and the energy efficiency [13-15]. In [13], the quadruped robot learned symmetric, speed-appropriate and low-energy gaits patterns with RL in the simulation. Hwangbo *et al*. proposed to directly train a policy based on RL to generate the corresponding energy-efficient gait patterns with different speed commands. An actuator net was trained using supervised learning to simulate a real motor performance to achieve better motion when the trained network was directly transferred to a real robot [14]. Fu *et al*. trained three fixed-speed policies using the three different speeds to generate the corresponding energy-saving gait patterns [15]. The joint positions obtained directly using RL are not limited by artificially planned movements and have a better performance on energy consumption. However, the desired speed was not dynamic during the motion. It did not reflect the dynamic adaptability of the policy for different speeds. What's

Manuscript received December 27, 2022; revised April 2, 2023, May 14, 2023, and July 2, 2023; accepted July 17, 2023. Recommended by Associate Editor Yongping Pan under the direction of Senior Editor Jongeun Choi. This work was supported by the Science and Technology Program of Beijing Municipal Science and Technology Commission (Z211100004021021).

Tiantian Hao, De Xu, and Shaohua Yan are with CAS Engineering Laboratory for Intelligent Industrial Vision, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China and the School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China (e-mails: {haotiantian2020, de.xu, yanshaohua2019}@ia.ac.cn).
* Corresponding author.

©ICROS, KIEE and Springer 2024

more, the leg movements obtained directly by RL are unsmooth. Although this problem can be solved by carefully designing the reward function [16], the tedious work of adjusting the reward function still does not really reflect the advantages of the learning-based approach. An end-to-end RL also has a slow speed to learn the gait patterns. Although the training environment developed by Nvidia achieved a fast policy learning for the real-world robotic tasks by using massive parallelism on a single GPU, there is no solution to this problem in principle [17]. To improve the efficiency of RL, it is possible to combine RL with traditional control methods. Yang *et al*. proposed to combine the evolution strategies as a high-level policy with the low-level convex MPC controller to generate gait patterns according to the speeds to maximize energy efficiency [18]. This kind of hierarchical control framework is not only able to ensure the stability of the robot's motion by the traditional control method, but also able to improve the adaptability of the robot's motion by the learning-based method.

The above mentioned methods achieve energy saving by generating different gait patterns at different speeds. In this paper, we further investigate how to achieve energy-saving motion with a planned gait pattern. To the best of our knowledge, this article is the first work to discuss how to adjust gait parameters to achieve energy-efficient motion with a planned gait. The proposed method is a hierarchical control framework. The low-level controller is virtual model control (VMC) which is an intuitive control method. The high-level controller is a gait modulator based on RL to adjust the gait parameters without changing the gait pattern. The main contributions of this article are as follows:

1) This article proposes an quadrupedal locomotion framework which combines with RL to achieve energy-saving motion. Compared with the traditional motion control, the proposed motion control is more flexible, energy-efficient and stable.
2) A gait modulator using RL is proposed in this article. It is able to adjust the gait parameters with a planned gait pattern according to the different speeds. The modulator increases the adaptability of the motion control, meanwhile maintains the performance of velocity tracking.

2. MODEL ARCHITECTURE AND CONTROL PRINCIPLE

2.1. Model architecture

The architecture of quadrupedal locomotion framework in an energy-efficient way is shown in Fig. 1 . The locomotion of the quadruped robot can be abstracted as the motion planning of four parallel mechanisms with a kinetic base. The whole motion planning comprises the ends' trajectory planning of the swing legs and the force planning of the stance legs. The state of each leg turns between swing and stance during the motion. Thus, the framework mainly consists of three parts: a gait generator, swing leg control and stance leg control. The gait generator is a high-level planner which includes a gait scheduler and a gait modulator. The gait scheduler generates the planned gait according to the handcrafted gait parameters. To improve the adaptability of the gait generator, this paper proposes a gait modulator based on RL to adjust the gait parameters for the energy-saving motion. The leg states are divided into swing and stance. Different control methods are used for the different states. In the swing leg control module, the position control method is adopted for the swing leg. In the stance leg control module, the torque control method is adopted for the swing leg.

2.2. Control principle

The gait pattern is expressed as gait parameters and is described in the third section. The control principle and

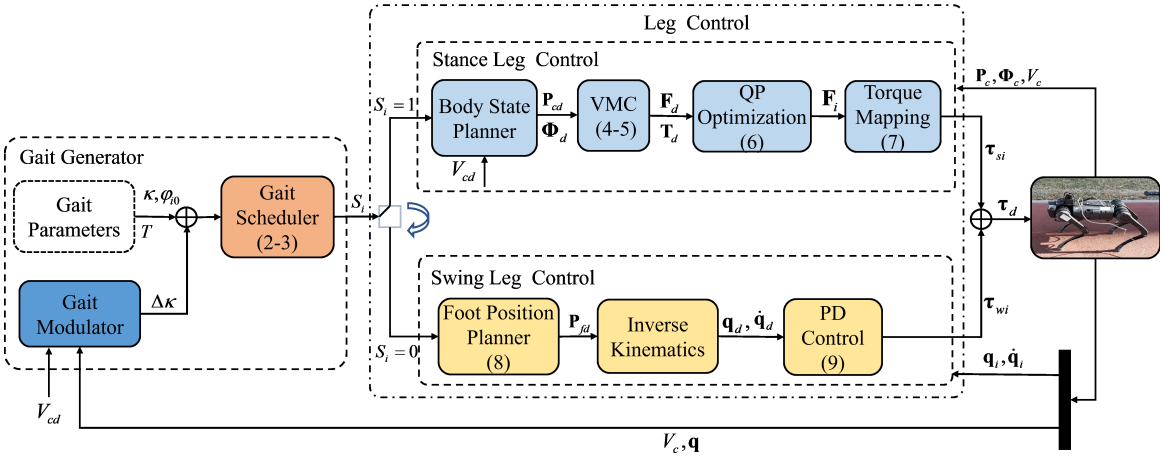


Fig. 1. The architecture of quadrupedal locomotion framework in an energy-efficiently way.

symbols referred in Fig. 1 are as follows. The gait modulator generates the increment of stance ratio $\Delta\kappa$ based on RL according to the state which includes the desired body velocity V_{cd}, joint angles $\mathbf{q}$ and the actual body velocity V_c to adaptively adjust the gait parameters. The gait scheduler maps the adjusted gait parameters into the desired states of the legs S_i. For the swing legs, the feet positions $\mathbf{P}_{fd}$ are planned in task space and are translated to joint space to track the desired joint positions $\mathbf{q}_d$ by inverse kinematics. Then proportional-derivative (PD) controller is used to generate the joint torques τ_{wi} . For the stance legs, the desired body position $\mathbf{P}_{cd}$ is calculated based on the desired body velocity V_{cd}. VMC is used to model the dynamics of the robot to get the resultant force $\mathbf{F}_d$ and resultant torque $\mathbf{T}_d$. Then $\mathbf{F}_d$ and $\mathbf{T}_d$ are decomposed to the component of force $\mathbf{F}_i$ applied to the feet of the stance legs with quadratic programming (QP) optimization. $\mathbf{F}_i$ is mapped to the joint torques τ_{si} by Jacobian matrix. Finally, the twelve joints' torques τ_d as the control signals are sent to the robot.

3. GAIT GENERATOR

3.1. Gaits pattern

The gait pattern is determined by the period of motion T, the stance ratio κ of each leg during the period of motion which is the ratio of stance time to the period of motion T, and the initial normal phases φ_{i0} (i = FL, FR, HL, HR) of the four legs. FL, FR, HL and HR represent the left front leg, right front leg, left hind leg and right hind leg, respectively. The period of motion T is the sum of the swing time and the stance time. There are two methods to obtain the value of the stance ratio κ. The first method uses (1) to compute the value of κ [19]. For the trot gait, the duration of the stance phase is nearly equal to the duration of the swing phase, so the value of stance ratio is usually set near 0.5 [20]. In the second method, RL is employed to obtain the optimal value of the stance ratio for a specific locomotion task [18]. The initial normal phases are used to determine the order of the four legs when to swing during the motion. In other words, the first two parameters, T and κ, represent the periodic characteristics of the gait pattern, while the initial normal phases represent the essential differences between the gait patterns. For the convenience of this study, the right front leg is set as the reference leg, i.e., its initial normal phase is 0. The initial normal phases of the other three legs for a planned gait pattern can be determined according to their lag phases with respect to the reference leg. During the motion, the normal phase of each leg can be expressed as (2). Stance ratio not only is an important parameter to obtain the stance time of one leg in a period of motion, but also serves as a threshold of the phase to get the desired states of the four legs at the next time step. The desired state of each leg can be expressed as (3). If the normal phase of one leg is less than the threshold, its desired state at the next time step is standing, otherwise it is swing. The legs of the quadruped robot are in periodic states based on the prescribed gait parameters.

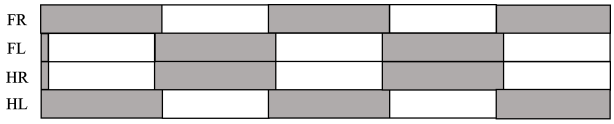


Fig. 2. The timing diagram of the trot gait. The white blocks indicate the legs in the swing phase and the gray blocks indicate the legs in the stance phase.

$$\kappa = \frac{t_s}{T}, \tag{1}$$

where t_s is the duration of the stance phase during the motion period T.

$$\varphi_i(t) = mod(t + \varphi_{i0}T, T)/T, \tag{2}$$

where $\varphi_i(t)$ is the normal phase of ith leg whose value is in the range [0, 1). t is time. mod is the modulo division function, its result is the remainder.

$$S_i = \begin{cases} 1, & 0 \le \varphi_i(t) < \kappa, \\ 0, & \kappa \le \varphi_i(t) < 1, \end{cases} \tag{3}$$

where S_i is the state of the ith leg which is standing if it is 1 and swing otherwise.

3.2. Trot generation

The trot is a kind of common gait in quadrupeds, so we describe in detail how to generate such a gait using the above gait parameters. A timing diagram of the trot gait is drawn to illustrate the gait characteristics intuitively, as shown in Fig. 2. The white blocks indicate the legs in the swing phase and the gray blocks indicate the legs in the stance phase. The trot has a bigger ratio of stance time to swing time for each leg, i.e., $\kappa = 0.6$. Because the trot is a type of the two-beat gait that the motion of the diagonal legs is consistent, the initial normal phases of the legs are 0, 0.5, 0.5 and 0, respectively. When the quadruped robot starts to move, the normal phases and the states of the legs are obtained from (2) and (3). At the beginning, the four legs are all in the case of standing. Then FL and HR start to swing, while FR and HL are standing. Before FR and HL start to swing, there is a period of stance phase for four legs. The gait pattern of the trot represents the swing sequence of FL(FR) - HR(LR).

4. LEG CONTROL

4.1. Stance leg control

The stance legs not only maintain the stability of the body, but also provide the force required for the body to

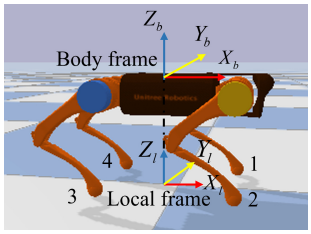


Fig. 3. The coordinate system.

move forward. The VMC is an intuitive control method that does not require a complex dynamics model, but only the construction of static equilibrium equations [21]. The coordinate system used in the leg control is shown in Fig. 3. For convenience, FL, FR, HR and HL are numbered 1, 2, 3, and 4, respectively. The quadruped robot is considered as a single rigid body. The desired resultant force and torque are calculated using the impedance controller by constructing the virtual spring and damping, as shown in (4) and (5).

$$\mathbf{F}_d = \mathbf{K}_{fp}(\mathbf{P}_c - \mathbf{P}_{cd}) + \mathbf{K}_{fd}(\dot{\mathbf{P}}_c - \dot{\mathbf{P}}_{cd}) + M\mathbf{G}, \tag{4}$$

$$\mathbf{T}_d = \mathbf{K}_{tp}(\mathbf{\Phi} - \mathbf{\Phi}_d) + \mathbf{K}_{td}(\dot{\mathbf{\Phi}} - \dot{\mathbf{\Phi}}_d), \tag{5}$$

where $\mathbf{F}_d = [F_x, F_y, F_z]^{\mathrm{T}}$ is the desired resultant force vector, $\mathbf{P}_c$ is the position vector of center of mass in body frame. $\mathbf{T}_d = [T_x, T_y, T_z]^{\mathrm{T}}$ is the desired resultant torque vector. $\mathbf{\Phi}$ is the attitude angles of the body, i.e., roll, pitch, and yaw. The subscript d indicates the desired value. $\mathbf{G} = [0, 0, -9.8]^{\mathrm{T}}$ m/s^2 is a gravitational acceleration vector. M is the mass of the robot. $\mathbf{K}_{fp}$, $\mathbf{K}_{fd}$, $\mathbf{K}_{tp}$ and $\mathbf{K}_{td}$ are the gains.

The next step is to decompose the resultant force and resultant torque to each stance leg. We take two legs supporting at the same moment as an example. The resultant force $\mathbf{F}_d$ and resultant torque $\mathbf{T}_d$ obtained from (4) and (5) are used to compute the support force for each stance leg, as given in (6). Then we can use the QP optimization method to optimize the decomposition of the above control quantities. When the first foot and third foot are stance, the reaction forces are as follows:

$$\begin{bmatrix} \mathbf{I}_{3\times3} & \mathbf{I}_{3\times3} \\ \mathbf{P}_{c1} & \mathbf{P}_{c3} \end{bmatrix} \begin{bmatrix} \mathbf{F}_1 \\ \mathbf{F}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{F}_d \\ \mathbf{T}_d \end{bmatrix}, \tag{6}$$

where $\mathbf{F}_1 \in \mathbb{R}^3$, $\mathbf{F}_3 \in \mathbb{R}^3$ are the reaction force of the first foot and third foot. $\mathbf{P}_{c1} = \mathbf{P}_c - \mathbf{P}_{f1}$, $\mathbf{P}_{c3} = \mathbf{P}_c - \mathbf{P}_{f3}$, $\mathbf{P}_{f1} \in \mathbb{R}^3$, $\mathbf{P}_{f3} \in \mathbb{R}^3$ are the positions of the first foot and third foot in body frame. $\mathbf{I}_{3\times3}$ is an identity matrix.

Similarly, the reaction forces for the second foot and forth foot are computed as above when they are stance. Finally, the obtained foot forces are multiplied with the corresponding Jacobian matrix of the leg to obtain the joint torques as given in (7).

$$\tau_{si} = -\mathbf{J}_i^{\mathrm{T}} \mathbf{F}_i, \tag{7}$$

where τ_{si} is the joint torques of the ith leg, which are transferred to the motors' torques with the software provided by the manufacture company of the quadruped robot. $\mathbf{J}_i$ is obtained by the traditional method [22]. The torques in (7) are the joint torques.

4.2. Swing leg control

The motion of the swing legs is the main reason why the quadruped robot has the ability to cross obstacles during the motion and adapt to rough environments better than wheeled robots. The linear interpolation method is used to generate the foot trajectory in the air based on the lifting and landing position. The cycloid function is chosen as the curve equation of the swing foot trajectory as given in (8). The landing position is calculated using the Raibert Heuristics method [23]. The desired position of the foot in the air is mapped to the joint angles of the leg using inverse kinematics. Then PD controller is used as the swing leg controller to track the designed trajectory. The motors' torques are computed as given in (9). The leg joint is taken as a first order inertial element and a first order integral element in series for position control. The control system with the PD controller is stable if the controller's parameters are adequately evaluated.

$$\begin{cases} x_d = (x_e - x_b)(\lambda - \sin\lambda)/(2\pi) + x_b, \\ y_d = (y_e - y_b)(\lambda - \sin\lambda)/(2\pi) + y_b, \\ z_d = h_d(1 - \cos\lambda)/2 + z_b, \end{cases} \tag{8}$$

$$\tau_{wi} = \mathbf{K}_{sp}(\mathbf{q} - \mathbf{q}_d) + \mathbf{K}_{sd}(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d), \tag{9}$$

where (x_b, y_b, z_b), (x_e, y_e, z_e), (x_d, y_d, z_d) are the lifting, landing and desired position of the swing foot in local frame, respectively. The landing position (x_e, y_e, z_e) is computed according to the velocity of the motion and the swing time. $\lambda = 2\pi t/t_w, t \in [0, t_w]$ is the real-time time of the swing and is reset to 0 at the moment of lifting. h_d is the desired lifting height. $\mathbf{q}$, $\dot{\mathbf{q}}$ are joint angles and angular velocity. The subscript d indicates the desired value. τ_{wi} is the motors' torques of the ith leg and $\mathbf{K}_{sp}$, $\mathbf{K}_{sd}$ are the gains.

5. GAIT MODULATOR

The role of the gait modulator is to adjust the gait parameters appropriately according to the state of the robot to achieve the energy-saving motion. This task can be modeled as a Markov decision process (MDP), which is quite suitable for RL. A MDP is defined by a tuple $\{s_t, a_t, r(s_t, a_t), P(s_{t+1} \mid s_t, a_t)\}$. A transition function $P(s_{t+1} \mid s_t, a_t)$ is the probability density of s_{t+1} given state s_t and action a_t. A reward function $r(s_t, a_t)$ implies how successful the action at corresponding to the state s_t is. A policy $\pi(a_t \mid s_t)$ chooses the action a_t according to the state s_t. The goal of RL is to learn a policy π to maximize the expected total reward at the end of episode, as given in (10).

$$R_T = \sum_{t=1}^{n} r_t(s_t, a_t, s_{t+1}), \tag{10}$$

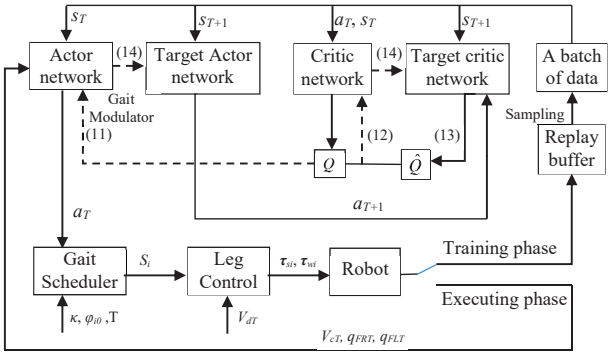


Fig. 4. The workflow of DDPG in the network training of RL for the locomotion of the quadruped robot.

where n denotes the time steps of each episode.

The trot gait with the fixed gait parameters may be not suitable for the different velocities during the motion. What's more, it is also difficult to achieve the energy-saving motion. When the gait pattern is planned, the value of stance ratio needs to be adjusted according to the velocity for an gait. In this paper, a gait modulator based on RL is proposed to achieve an energy-saving trot gait.

A method for adjusting the stance ratio using deep deterministic policy gradient (DDPG) [24] is proposed. DDPG is a model-free and off-policy actor-critic RL algorithm for the continuous action domain. The workflow of DDPG in the network training of RL for the locomotion of the quadruped robot is shown in Fig. 4. It involves the training phase and the executing phase. The training phase is in the simulation. The policy function called the actor is estimated by a neural network. The value function called the critic is also estimated by a neural network . The data used to train the neural networks are obtained by interacting with the simulation environment. The learned actor network as the gait modulator is used in the executing phase to obtain the increment of the stance ratio $\Delta\kappa$. The desired joint torques of the real robot are obtained in Section 4. We parameterize the actor network and the critic network by and θ^{μ} and θ^{Q}. Similarly, the target networks for the actor network and the critic network are parameterized by $\theta^{\bar{\mu}}$ and $\theta^{\bar{Q}}$. We update the action distribution of the actor network by applying policy gradient [24]:

$$\nabla_{\theta^{\mu}} J \approx \frac{1}{m}\sum_{i}\left[\nabla_{\theta^{\mu}} Q(s,a\,|\,\theta^{Q})\,|_{s=s_i,a=\mu(s_i|\theta^{\mu})}\right], \quad (11)$$

where $\nabla_{\theta^{\mu}}$ is the gradient of the policy's parameters. m is the number of transitions (s_i,a_i,r_i,s_{i+1}) in a batch of data.

The loss function is used to update critic network.

$$L(\theta^{Q}) = \frac{1}{m}\sum_{i}(\hat{Q}_i - Q(s_i,a_i\,|\,\theta^{Q}))^2, \quad (12)$$

$$\hat{Q}_i = r_i + \gamma\bar{Q}(s_{i+1},\bar{\mu}(s_{i+1}\,|\,\theta^{\bar{\mu}}))\,, \quad (13)$$

where $Q(s_i,a_i\mid\theta^{Q})$ is the estimated the action-value function and $\bar{Q}(s_{i+1},\bar{\mu}(s_{i+1}\mid\theta^{\bar{\mu}}))$ is the target action-value function, which is the output of the target network. r_i is the reward for the action a_i and γ is a discount factor.

The target networks are updated by tracking the learned networks slowly with $\zeta \ll 1$ to improve the stability of learning, as given in (14).

$$\begin{cases}\theta^{\bar{Q}} \leftarrow \xi\theta^{Q} + (1-\xi)Q^{\bar{Q}},\\ \theta^{\bar{\mu}} \leftarrow \xi\theta^{\mu} + (1-\xi)Q^{\bar{\mu}}.\end{cases} \quad (14)$$

The experimental experiences tell us that it will cause the instability when the gait parameters are changed in the period of motion. Thus, the high-level controllers work at a low frequency of $1/T$ Hz. When each period of motion ends, the gait modulator generates the action according to the state during the cycle. To reduce the dimension of the state, the essential information is involved at each period of motion. In trot gait, the joint angles of FR (FL) are the same with HL (HR). What's more, the thigh angle is obviously influenced by the action. Thus, FR's thigh angle q_{FR} and FL's thigh angle q_{FL} are parts of the state. The desired and actual velocity, i.e., V_d and V_c, are directly related with the action and indirectly related with the body position and the joint torques. Thus, V_c and V_d are included, while the body position and the joint torques are not included in the state. To avoid excessive state dimensionality, 10 state points are sampled at medium intervals from each period of motion to form a 40-dimensional state $s_T = \{V_{c1}, V_{d1}, q_{FR1}, q_{FL1}, ..., V_{c10}, V_{d10}, q_{FR10}, q_{FL10}\}$. The action $a_T = \{\Delta\kappa\}$ represents the increment of stance ratio and acts in the entire period of motion, as given in (15). The next state s_{T+1} is recorded after a_T has been executed during the period of motion. The action a_T from the actor network is continuous. Thus, the stance ratio can be obtained by $\kappa = \kappa_0 + \Delta\kappa$ where κ_0 is the initial stance ratio. In order to avoid the action making the robot produce dangerous or meaningless motion. The value of the action is limited within a handcrafted suitable range.

$$\Delta\kappa = \pi(s_T), \quad (15)$$

where π is the policy.

To encourage the agent to learn stable and energy-efficient gait parameters, the reward function involves the energy reward and the attitude angles reward. During the training, the object of the agent is to get maximum reward. In other words, the cost of energy and the deviation of the attitude angles are all small. The reward function at sampling time t is given in (16).

$$\begin{cases} r_t = w^{en}r_t^{en} + w^{eu}r_t^{eu},\\ w^{en} = 1,\ w^{eu} = 5,\end{cases} \quad (16)$$

where the energy reward r_t^{en} is used to measure the energy consumption in terms of joint torques and the increment

of the joint angles [25]. The attitude reward r_t^{eu} expects to maintain the stability of the body during motion.

$$r_t^{en} = -\sum_{k=1}^{12} |\tau_{kt}(q_{kt} - q_{k(t-1)})|, \tag{17}$$

$$r_t^{eu} = -|\phi_t - \phi_{t-1}| - |\beta_t - \beta_{t-1}|, \tag{18}$$

where τ_{kt} is the kth joint torque at sampling time t. q_{kt} and $q_{k(t-1)}$ are the kth joint angle at sampling time t and $t-1$. ϕ_t and ϕ_{t-1} are the roll angles of the robot body at sampling time t and $t-1$. β_t and β_{t-1} are the pitch angles of the robot body at sampling time t and $t-1$.

The low-level controllers ensure that the actual velocity of the robot tracks the desired velocity. Thus, the error between the actual velocity and desired velocity and the deviation of the yaw angle are not involved in the reward function. At the end of the period of motion, the cumulative reward r_T is calculated. The pair $\{s_T, a_T, r_T, s_{T+1}\}$ is stored in a replay buffer. Then a batch of the pairs is sampled from the replay buffer to train the networks.

6. EXPERIMENTS AND RESULTS

6.1. Training details

The policy network in the gait modulator is trained using DDPG which has been demonstrated multiple times for robot motion control tasks [26,27]. The policy network is a 4-layer multilayer perceptron (MLP) with 512 nodes in the hidden layer. Activation unit in the policy and value network is ReLU. We use sigmoid activation for the policy network's last layer and linear activation for the value network's last layer. The networks are updated using Adam with a learning rate of 1×10^{-4} and a batch size of 256 transitions. The size of replay buffer which stores the samping data is 1×10^6. The value of the discount factor is set to 0.99 and the soft update rate is 5×10^{-3}. The action from the policy network is in the range [0, 1]. The specified amplitude is 0.3. Then the obtained increment stance ratio is in the range [0, 0.3].

6.2. Simulation

We conducted the simulation experiments using the A1 robot from Unitree in Pybullet simulator. The simulation time of the simulator was set to 2 ms. The trot gait parameters are listed in Table 1. These parameters are fixed values except for the stance ratio. The stance ratio will be determined by the policy network within this range. A motion task was designed to train the agent. The desired velocity command is usually varied discretely during the motion,

Table 1. The parameters of trot gait.

Parameter	Gait cycle(s)	Initial normal phase	Stance ratio
Value	0.4	0, 0.5, 0.5, 0	$0.4 \sim 0.7$

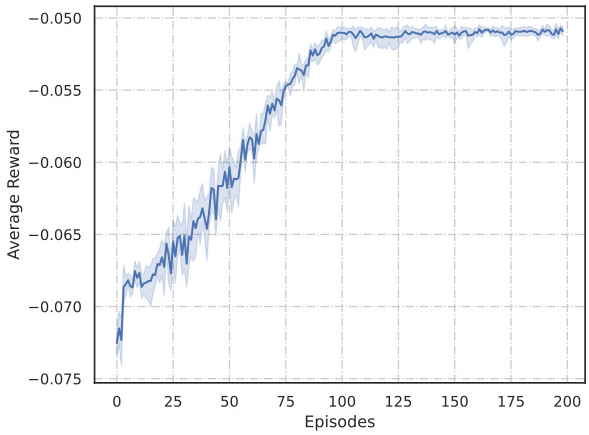
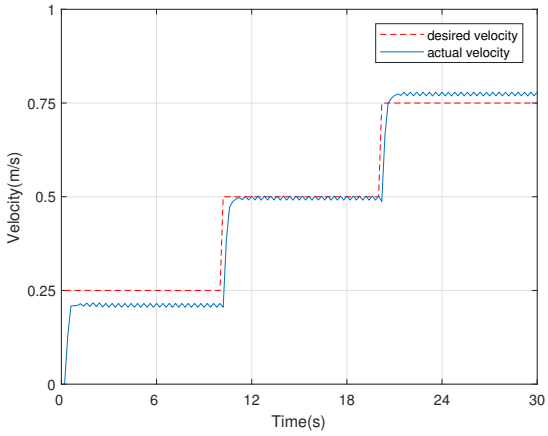


Fig. 5. The average reward during training.

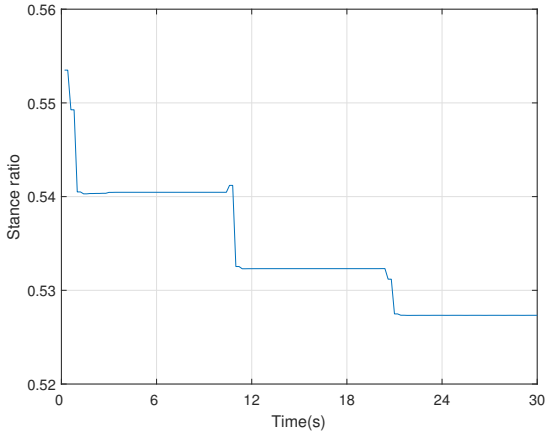
so three different desired velocity commands were set in each episode during training which were 0.25 m/s, 0.5 m/s and 0.75 m/s, respectively. Each velocity command was held for 5000 time steps, i.e., 10 s. In order to get a robust policy, domain randomization was incorporated into the training process [28]. During training, when the robot falls down or successfully completes the required motion task, one episode is end. We trained the agent with five different random seeds. The average reward curve during training is shown in Fig. 5. The curve shows that the proposed method requires about 100 episodes to learn a stable policy.

To test the performance of the learned policy, the robot trotted with the stance ratio containing the increment obtained from the learned policy in the simulation. The initial value of the stance ratio is 0.4, the increment of the stance ratio is 0.153, thus the stance ratio at start of cycle is 0.553. The actual velocity of the robot was recorded at the end of each period of motion. The velocity curves and the values of κ are shown in Fig. 6. It can be found that the robot is able to adjust its own velocity in time according to the desired velocity. When the desired velocity is fixed, the actual velocity of the robot also keeps stable. When the desired velocity changes, the actual velocity tracks the desired velocity immediately. The error of the velocity tracking is less than 0.045 m/s. Velocity error is very small and the integral controller affects the real-time performance, thus the integral controller is not added. The gait modulator using RL adjusts the value of κ according to the velocity. As the desired velocity increases, the value of κ decreases to the appropriate value which fits the desired velocity well and keeps constant when the desired velocity is fixed. In other words, the stance time becomes shorter in a period of motion for the high-speed motion. This is an important condition to ensure the velocity tracking and the stability of the motion.

The comparative simulations are well conducted to



(a) Velocity.



(b) Stance ratio κ.

Fig. 6. The velocity and stance ratio κ during motion.

verify the energy-efficient performance of the proposed method. We chose fixed parameter method (FPM) and Yang's method [18] (YM) which also uses a hierarchical control framework as the comparative methods. In FPM, the value of the stance ratio was set to 0.5. In YM, the action obtained from the policy network was the same with our method and VMC was used to control the stance leg for a fair comparison. The accumulated energy consumption of each method at different phases was calculated by (19). The sum of the energy consumption of the robot was recorded every 0.2 s.

$$E = \sum_{j=1}^{N_c} \sum_{k=1}^{12} |\tau_{kj}(q_{kj} - q_{k(j-1)})|, \tag{19}$$

where E is the accumulated energy consumption, τ_{kj} is the kth joint torque at sampling time j. q_{kj} and $q_{k(j-1)}$ are the kth joint angle at sampling time j and $j-1$. N_c is the sampling times.

The energy consumption curves are shown in Fig. 7. It

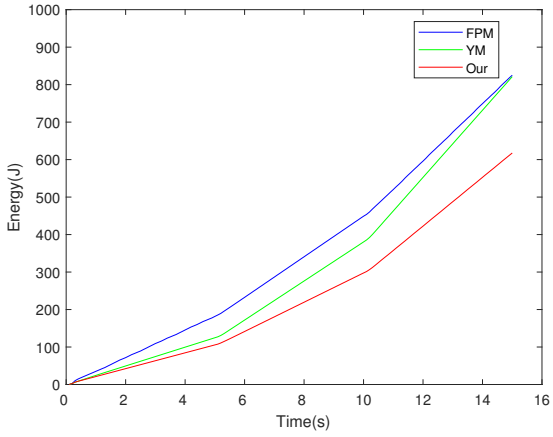


Fig. 7. The energy consumption curves of comparative simulations.

Table 2. The accumulated energy consumption in the three phases of comparison simulations.

Method	Energy consumption (J) under different velocities (m/s)		
	0.25	0.5	0.75
FPM	185	268	379
YM	127	258	444
Our	107	193	322

can be seen from the figure that the energy consumption at per sampling time is approximately the same in the same velocity phase for each method. Moreover, from the beginning of the motion, our method consumes the least energy compared with the other two methods. The accumulated energy consumption at each velocity phase is shown in Table 2. It can be found that FPM consumes the most energy in the 0.25m/s and 0.5m/s phases during the motion, and the other two methods based on the hierarchical control framework are able to adjust the stance ratio according to the velocity to reduce the energy consumption. However, YM costs the most energy in the 0.75 m/s phase. It demonstrates that YM has the limitation at high velocity. In contrast, our method consumes the least energy which is 57.8%, 72.0% and 84.9% of FPM and 84.2%, 74.8% and 72.5% of YM. Overall, when the robot moves using a planned gait pattern, the energy consumption increases as the velocity increases. The gait with a variable stance ratio according to the velocity can reduce the energy consumption to a certain extent. This advantage will be more prominent if the robot works for a long time.

6.3. Experiments on A1 robot

We deployed the trained network directly to the A1 robot. The experimental system is shown in Fig. 8. During the experiment, the control code was run on a personal

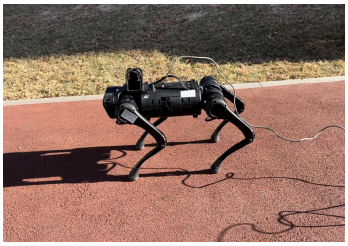


Fig. 8. The experimental system.

Table 3. The gains used in the experiments.

Gains	$\mathbf{K}_{fp}$	$\mathbf{K}_{fd}$	$\mathbf{K}_{tp}$	$\mathbf{K}_{td}$	$\mathbf{K}_{sp}$	$\mathbf{K}_{sd}$
Diagonal values	10	6.3	1000	63	10	0.1
	10	6.3	1000	63	10	0.2
	200	20	1000	63	7	0.2

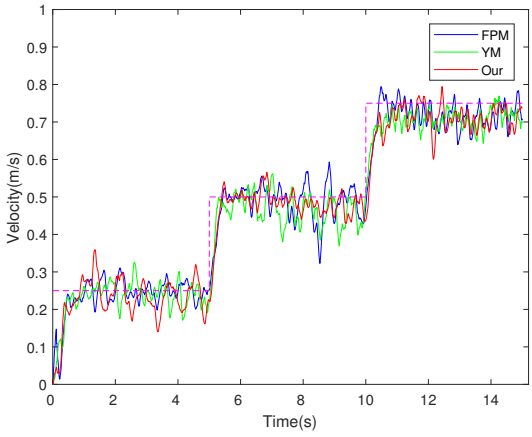


Fig. 9. The velocity tracking curves during the movement using different methods.

laptop. The states of the robot were read and the control signals were sent by a network cable. The low-level controllers ran at a frequency of 500 Hz and the high-level controllers ran at a frequency of 2.5 Hz due to the period of motion is 0.4 s.

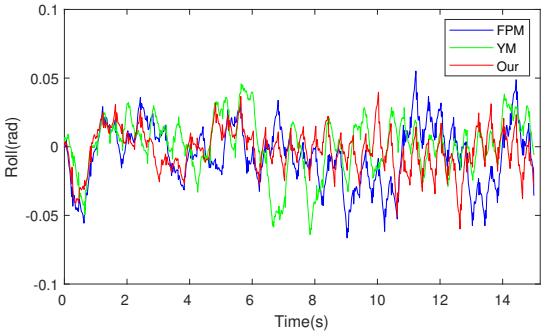
We conducted the comparative experiments on the real robot in order to comprehensively evaluate the energy-saving performance of different methods. The gains used in the experiment are listed in Table 3. The values of the gains were initially determined using the Ziegler-Nichols method [29] in the simulation and later tuned on the real robot to ensure the safety and stability of the robot. In the experiment, each velocity phase lasted 5 s, then the energy consumption of FPM, YM and our method were recorded on the real robot. The velocity tracking curves during the movement are shown in Fig. 9. The carmine dotted line is the desired velocity. All three methods are able to track the desired velocity. The distance of the body movement is 7.383 m, 7.423 m and 7.385 m, respectively. They are very close. It means that their performances of the tracking are similar.

Table 4. The accumulated energy consumption in the three phases of comparison experiments on A1.

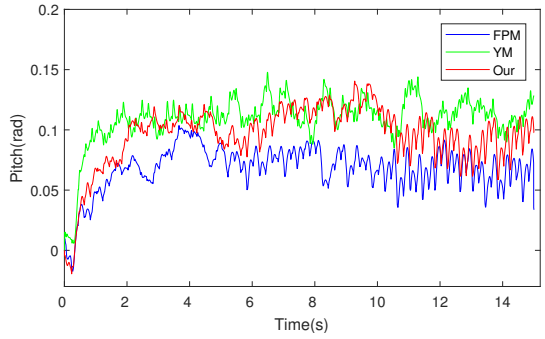
Method	Energy consumption (J) under different velocities (m/s)		
	0.25	0.5	0.75
FPM	330	383	462
YM	404	476	610
Our	316	373	446

The energy consumption with different methods is shown in Table 4. It can be found that our method consumes the least energy in three velocity phases. The energy consumption decreases by 4.2%, 2.6%, 3.5% of FPM and 21.8%, 21.6%, 26.9% of YM in different velocity phases. Unlike the simulation results, YM consumes the most energy in three phases. YM was originally designed to generate different gait patterns according to different speeds to decrease the energy consumption, but there is no guarantee that the gait parameters generated in the same gait pattern are optimal. Moreover, the desired and actual velocity at sampling time as the state are not enough. Compared with the simulation results, the real robot has shorter working hours, but it consumes more energy. The mechanical characteristics of the motor in the simulation are not the same with the real robot. The real robot will consume more energy during the motion, especially in the initial start-up phase and the acceleration phase. In addition, the parameters of the real environment are different from those in the simulation environment such as the friction force of the actual road surface and the type of the terrain.

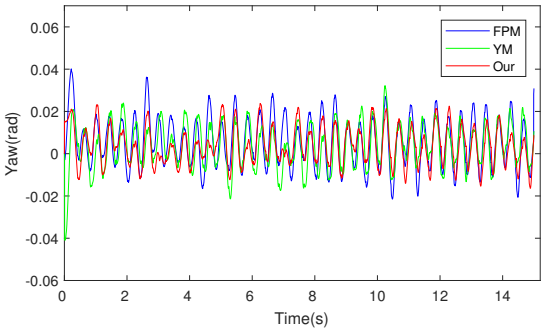
To test the stability of the robot during the motion, the attitude angles were recorded. The attitude angles curves are shown in Fig. 10 and the average attitude angle errors of comparison experiments are listed in Table 5. The desired attitude angles are all 0. It can be seen from Fig. 10 that the roll angle fluctuates around the desired value during the motion. The amplitude increases when the velocity increases. The pitch angle of each method fluctuates around a fixed positive value and the frequency of oscillation increases when the velocity increases. The yaw angle varies at a fixed frequency. The pitch angle is not as important as the roll and yaw angles. Sometimes the robot will adjust the pitch angle to ensure the performance of the velocity tracking. It can be found that the average attitude angle errors of our method decrease by 25.0%, 11.9% and 11.1% of YM. The average roll and yaw errors of our method decrease by 33.3% and 27.3% of FM. The pitch error for the proposed method is acceptable and approximately same as the result of YM. In summary, our method can achieve both energy-saving and stable motion.



(a) Roll.



(b) Pitch.



(c) Yaw.

Fig. 10. The attitude angles during the movement using different methods.

Table 5. The average attitude angle errors of comparison experiments on A1.

Method	Average roll error (rad)	Average pitch error (rad)	Average yaw error (rad)
FPM	0.018	0.068	0.011
YM	0.016	0.109	0.009
Our	0.012	0.096	0.008

7. CONCLUSION

This paper is the first attempt to adjust the value of the stance ratio in gait parameters to achieve stable and energy-efficient motion of a quadruped robot with a planned gait. A gait modulator using RL is proposed to achieve the role of parameter adjustment according to the desired velocity. The energy-efficient performance of the proposed hierarchical control framework is verified in the simulation and on A1 robot. In the simulation, our method consumes the least energy which is 57.8%, 72.0% and 84.9% of FPM and 84.2%, 74.8% and 72.5% of YM in different velocity phases. On A1 robot, the energy consumption with our method decreases by 4.2%, 2.6%, 3.5% of FPM and 21.8%, 21.6%, 26.9% of YM in different velocity phases. In addition, the average attitude angle errors of our method decrease by 25.0%, 11.9% and 11.1% of YM. The average roll and yaw errors of our method decreased by 33.3% and 27.3% of FPM. It is demonstrated that our method has advantages in terms of energy efficiency and stability of motion compared with FPM and YM. The limitation of this paper is that the proposed energy-efficient method is mainly considered the situation working on the flat terrain.

In future, the transfer learning will be considered to reduce the gap between the simulation and real robot application with varying environments. In addition, a robust controller will be designed in order to enhance the robustness of robots against external disturbances and improve the adaptability to different terrains.

CONFLICT OF INTEREST

The authors declare that there is no conflict of interest.

REFERENCES

[1] A. Srisuchinnawong, B. Wang, D. Shao, P. Ngamkajornwiwat, Z. Dai, A. Ji, and P. Manoonpong, "Modular neural control for gait adaptation and obstacle avoidance of a tailless gecko robot," *Journal of Intelligent & Robotic Systems*, vol. 101, no. 2, pp. 27, 2021.

[2] C. D. Bellicoso, F. Jenelten, C. Gehring, and M. Hutter, "Dynamic locomotion through online nonlinear motion optimization for quadrupedal robots," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 2261-2268, 2018.

[3] D. Kim, J. D. Carlo, B. Katz, G. Bledt, and S. Kim, "Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control," arXiv preprint, arXiv: 1909.06586, 2019.

[4] Z. Fu, A. Kumar, J. Malik, and D. Pathak, "Minimizing energy consumption leads to the emergence of gaits in legged robots," *Proc. of the 5th Conference on Robot Learning, PMLR*, pp. 928-937, 2022.

[5] Z. Xu, Q. Zhuang, Y. Zhou, S. Lu, X. Wang, W. Cai, and Z. Zang, "Functional layers of inverted flexible perovskite solar cells and effective technologies for device commercialization," *Small Structures*, vol. 4, no. 5, pp. 2200338, 2023.

[6] D. F. Hoyt and C. R. Taylor, "Gait and the energetics of locomotion in horses," *Nature*, vol. 292 no. 5820, pp. 239-240, 1981.

[7] S. J. Wickler, D. F. Hoyt, E. A. Cogger, and G. Myers, "The energetics of the trot–gallop transition,"*Journal of Experimental Biology*, vol. 206, no. 9, pp. 1557-1564, 2003.

[8] I. M. Koo, T. D. Trong, Y. H. Lee, H. Moon, J. Koo, S. Park, and H. R. Choi, "Biologically inspired gait transition control for a quadruped walking robot," *Autonomous Robots*, vol. 39, no. 2, pp. 169–182, 2015.

[9] Y. Shao, Y. Jin, X. Liu, W. He, H. Wang, and W. Yang, "Learning free gait transition for quadruped robots via phase-guided controller," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1230-1237, 2022.

[10] A. Iscen, G. Yu, A. Escontrela, D. Jain, J. Tan, and K. Caluwaerts, "Learning agile locomotion skills with a mentor," *Proc. of IEEE International Conference on Robotics and Automation*, pp. 2019-2025, 2021.

[11] Y. J. Tang, J. Tan, and T. Harada, "Learning agile locomotion via adversarial training," *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 6098-6105, 2020.

[12] L. Smith, J. C. Kew, X. B. Peng, S. Ha, J. Tan, and S. Levine, "Legged robots that keep on learning: Fine-tuning locomotion policies in the real world," *International Conference on Robotics and Automation*, pp. 1593-1599, 2022.

[13] W. Yu, G. Turk, and C. K. Liu, "Learning symmetric and low-energy locomotion," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1-12, 2018.

[14] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, pp. 5872, 2019.

[15] Z. Fu, A. Kumar, J. Malik, and D. Pathak, "Minimizing energy consumption leads to the emergence of gaits in legged robots," *Proc. of the 5th Conference on Robot Learning*, pp. 928-937, 2022.

[16] G. Ji, J. Mun, H. Kim, and J. Hwangbo, "Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4630-4637, 2022.

[17] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," *Proc. of the 5th Conference on Robot Learning*, pp. 91-100, 2022.

[18] Y. Yang, T. Zhang, E. Coumans, J. Tan, and B. Boots, "Fast and efficient locomotion via learned gait transitions," *Proc. of the 5th Conference on Robot Learning, PMLR*, pp. 773-783, 2022.

[19] I. M. Koo, T. D. Trong, Y. H. Lee, H. Moon, J. Koo, S. Park, and H. R. Choi, "Biologically inspired gait transition control for a quadruped walking robot," *Autonomous Robots*, vol. 39, no. 2, pp. 169–182, 2015.

[20] K. Inagaki and H. Kobayashi, "A gait transition for quadruped walking machine," *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 525-531, 1993.

[21] G. Zhang, X. Rong, C. Hui, Y. Li, and B. Li, "Torso motion control and toe trajectory generation of a trotting quadruped robot based on virtual model control," *Advanced Robotics*, vol. 30, no. 4, pp. 284-297, 2016.

[22] Y. Xin, Z. Hong , B. Li, and Y. Li, "A comparative study of four Jacobian matrix derivation methods for quadruped robot," *Proc. of 34th Chinese Control Conference*, pp. 5970-5976, 2015.

[23] M. H. Raibert and E. R. Tello, "Legged robots that balance," *IEEE Expert*, vol. 1, no. 4, pp. 89-89, 1986.

[24] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," arXiv preprint, arXiv:1509.02971, 2015.

[25] K. Walas, D. Kanoulas, and P. Kryczka, "Terrain classification and locomotion parameters adaptation for humanoid robots using force/torque sensing," *Proc. of IEEE-RAS 16th International Conference on Humanoid Robots*, pp.133-140, 2016.

[26] Y. Ma, D. Xu, and F. Qin, "Efficient insertion control for precision assembly based on demonstration learning and reinforcement learning," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 7, pp. 4492-4502, 2021.

[27] J. Xu, Z. Hou, W. Wang, B. Xu, K. Zhang, and K. Chen, "Feedback deep deterministic policy gradient with fuzzy reward for robotic multiple peg-in-hole assembly tasks," *IEEE Transactions on Industrial Informatics*, vol. 15 no. 3, pp. 1658-1667, 2019.

[28] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 23-30, 2017.

[29] C. C. Hang, K. J. Astrom, and W. K. Ho, "Refinements of the Ziegler-Nichols tuning formula," *IEE Proceedings D: Control Theory and Applications*, vol. 138, no. 2, pp. 111-118, 1991.



Tiantian Hao received her B.Sc. degree in automation from Harbin Engineering University, Harbin, China, in 2020. She is currently pursuing a Ph.D. degree in pattern recognition and intelligent systems with the Institute of Automation, Chinese Academy of Sciences (IACAS), Beijing, China. She is also with the School of Artificial Intelligence, University of Chinese Academy of Sciences (UCAS), Beijing. Her current research interests include robot intelligent control and machine vision.



De Xu received his B.Sc. and M.Sc. degrees from the Shandong University of Technology, Jinan, China, in 1985 and 1990, respectively, and a Ph.D. degree from Zhejiang University, Hangzhou, China, in 2001, all in control science and engineering. He has been with the IACAS, since 2001, where he is currently a Professor. He is also with the UCAS. His current research interests include robotics and automation such as visual measurement, visual control, intelligent control, microscopic vision, microassembly, and skill learning.



Shaohua Yan received his B.Sc. degree in automation from Harbin Engineering University, Harbin, China, in 2019. He is currently pursuing a Ph.D. degree in pattern recognition and intelligent systems with the IACAS. He is also with the School of Artificial Intelligence, UCAS. His current research interests include robot intelligent control and machine vision.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.